

Inheritance, Polymorphism, and Composition

A Hands-On Guide Using the Springfield Elementary Class Hierarchy

Lab Overview & Objectives

This lab exercise explores core Object-Oriented Programming (OOP) principles in C++, including public inheritance, protected member access, runtime polymorphism via virtual functions, dynamic casting, and object composition. You will implement a robust class hierarchy representing various individuals at Springfield Elementary School.

Class Hierarchy Architecture:

- Person (Base Class): # name (string), # age (int) | Virtual display()
- Student (Derived from Person): # gpa (double) | Overrides display()
- Professor (Derived from Person): # yearsExperience (int) | Overrides display()
- Employee (Derived from Person): # ID (int) | Overrides display()
- Department (Composition): Holds department name, office number, and a collection of Professor pointers.

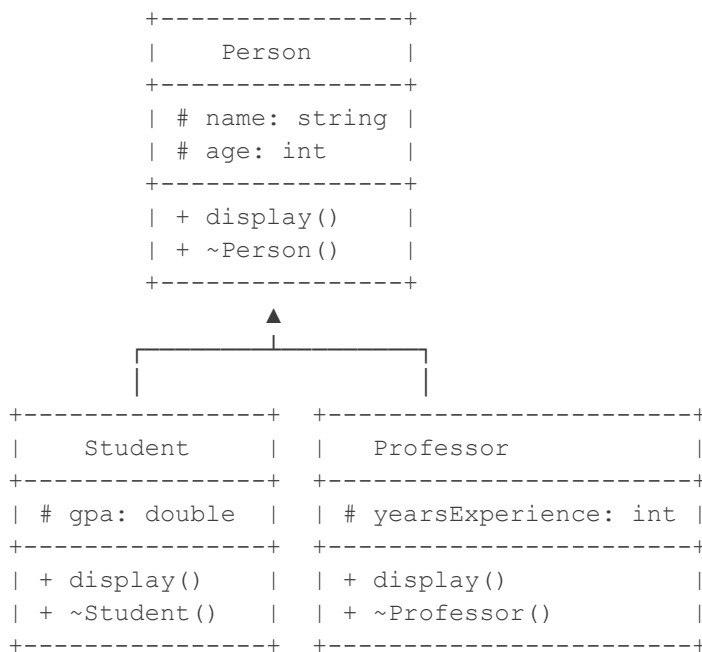


Figure 1. Partial UML diagram

Part 1 & 2: Creating the Base and Derived Classes

Implement the Person base class with protected attributes so that derived classes can access them directly. Then, implement the Student and Professor derived classes.

Part 3: Runtime Polymorphism

Declare display() as a virtual function in the Person class. This ensures that when calling display() through a base class pointer (Person*), the correct derived class method is invoked dynamically at runtime.

```
void experiment01() {
    cout << experiment01 " " << endl;
    Person* p1 = new Student("Bart Simpson", 10, 2.13);
    Person* p2 = new Professor("Edna Krabapple", 45, 28);
    p1->display(); // Calls Student's display()
    p2->display(); // Calls Professor's display()
    delete p1;
    delete p2;
}

int main() {
    experiment01();
    return 0;
}
```

Part 4: Advanced Inheritance & Casting

1. Virtual Destructors: Ensure that destructors in Person and all derived classes are declared virtual to prevent memory leaks when deleting derived objects via base pointers.
2. Vector of Person Pointers: Store mixed objects (Student, Professor, Employee) in a `std::vector<Person*>` and iterate through them to invoke polymorphic display functions.
3. `dynamic_cast<>`: Use `dynamic_cast` to safely inspect and identify specific derived types at runtime.

Part 5: Composition with Department

Create a Department class that encapsulates a department name, office location, and a collection (e.g., `vector<Professor*>`) representing faculty members assigned to the department.

Part 6 & 7: The Employee Class & Extended Test Suite

The Employee class inherits from Person and adds the employee ID attribute. Below is the complete extended test function to validate your implementation.

```
void experiment02() {
    cout << "experiment02 " << endl;
    // Part 3: Runtime Polymorphism
    cout << "=== Part 3: Runtime Polymorphism ===\n";
    Person* p1 = new Student("Bart Simpson", 10, 2.13);
    Person* p2 = new Professor("Edna Krabapple", 45, 28);
    p1->display();
    p2->display();
    delete p1;
    delete p2;

    // Part 4.2: Vector of Person Pointers
    cout << "\n=== Part 4.2: Vector of Person Pointers ===\n";
    vector<Person*> people;
    people.push_back(new Student("Milhouse Van Houten", 10, 2.85));
    people.push_back(new Professor("Elizabeth Hoover", 40, 15));
    people.push_back(new Employee("Groundskeeper Willie", 50, 45000.0));
    for (const auto& person : people) {
        person->display();
    }

    // Part 4.3: Testing dynamic_cast
    cout << "\n=== Part 4.3: Testing dynamic_cast ===\n";
    for (auto person : people) {
        if (Employee* emp = dynamic_cast<Employee*>(person)) {
            cout << "Found an Employee object!\n";
        }
    }

    // Part 5: Composition with Department
    cout << "\n=== Part 5: Composition Experiment ===\n";
    Professor* prof1 = new Professor("Seymour Skinner", 50, 20);
    Professor* prof2 = new Professor("Coach Krup", 42, 12);
    Department generalStudies("General Studies", "Building B, Room 101");
    generalStudies.addProfessor(prof1);
    generalStudies.addProfessor(prof2);
    generalStudies.display();

    // Clean up dynamic memory
    for (auto person : people) {
        delete person;
    }
}
```

```
    delete prof1;
    delete prof2;
}
```

Sample Output

```
experiment01
[Student] Name: Bart Simpson, Age: 10, GPA: 2.13
[Professor] Name: Edna Krabapple, Age: 45, Experience: 28
+ Student destructor called for Bart Simpson
+ Person destructor called for Bart Simpson
+ Professor destructor called for Edna Krabapple
+ Person destructor called for Edna Krabapple

experiment02
=== Part 3: Runtime Polymorphism ===
[Student] Name: Bart Simpson, Age: 10, GPA: 2.13
[Professor] Name: Edna Krabappel, Age: 45, Experience: 28
+ Student destructor called for Bart Simpson
+ Person destructor called for Bart Simpson
+ Professor destructor called for Edna Krabappel
+ Person destructor called for Edna Krabappel

=== Part 4.2: Vector of Person Pointers ===
[Student] Name: Milhouse Van Houten, Age: 10, GPA: 2.85
[Professor] Name: Elizabeth Hoover, Age: 40, Experience: 15
[Employee] Name: Groundskeeper Willie, Age: 50, ID: 1004

=== Part 4.3: Testing dynamic_cast ===
Found an Employee object!

=== Part 5: Composition Experiment ===
--- Department: General Studies (Office: Building B, Room 101) ---
[Professor] Name: Seymour Skinner, Age: 50, Experience: 20
[Professor] Name: Coach Krup, Age: 42, Experience: 12
+ Student destructor called for Milhouse Van Houten
+ Person destructor called for Milhouse Van Houten
+ Professor destructor called for Elizabeth Hoover
+ Person destructor called for Elizabeth Hoover
+ Employee destructor called for Groundskeeper Willie
+ Person destructor called for Groundskeeper Willie
+ Professor destructor called for Seymour Skinner
+ Person destructor called for Seymour Skinner
+ Professor destructor called for Coach Krup
+ Person destructor called for Coach Krup
all done!
```

Faculty & Staff at Springfield Elementary



Elizabeth Hoover, B.Ed.
(Lisa's Teacher)



Edna Krabapple



Principal Skinner



Coach Krup



Lunchlady Doris



Bus driver – Otto Mann



Groundskeeper Willie